Swift Duro® GPS Receiver

Last edited by **Kai Wang** Jan 17, 2026, 5:58 p.m. EST

This page is about the Swift Duro® GPS Receiver.



Setting up Swift Duro® GPS Receiver with NTRIP Service

NTRIP, or Networked Transport of RTCM via Internet Protocol, is a protocol for streaming differential GNSS corrections over the internet. Ingenuity Labs has a subscription from Cansel service. The connection information (URL, port, credentials and mountpoint) associated with the subscription can be found here.

Hardware

- Swift Duro GPS Receiver Unit (download user manual)
- PC/Laptop
- 12-24V DC Power Supply and Cable
- Ethernet Cable
- RS-232 Serial Cable (optional, for backup access to the GPS unit)

Software

- Swift Console (Available for both Windows and Linux, can be downloaded here)

Steps

> - The following steps are to be implemented on Ubuntu system.
> - When operating the GPS unit, make sure to be in an **outdoor, open area with minimal tall objects around** (e.g., trees, buildings).

1. Local Area Network Setup

Connect the GPS unit to your PC using an Ethernet cable. Set the following information for your PC:

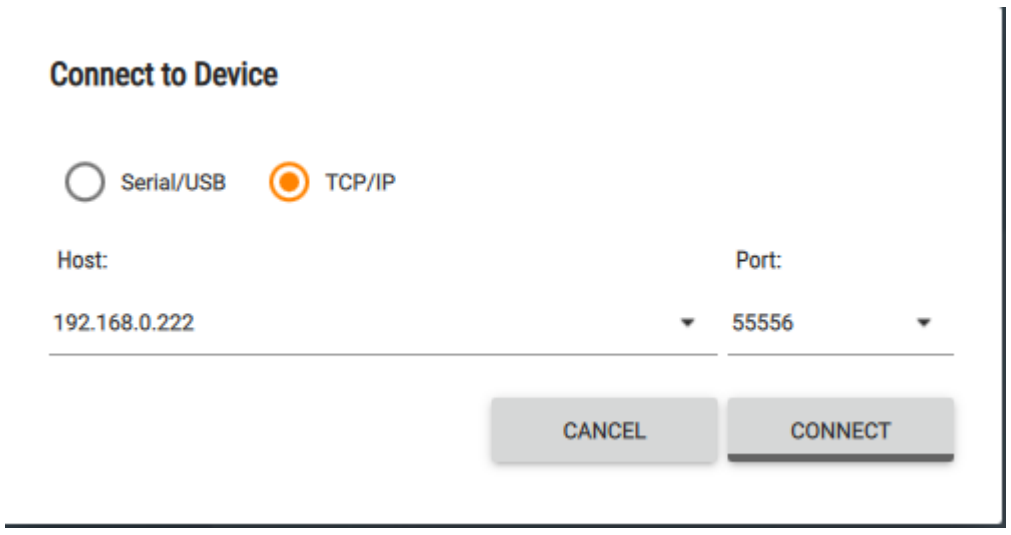
- IPv4 address: 192.168.0.10
- Subnet mask: 255.255.255.0

A quick and permanent way to do so in Ubuntu: open the .yaml file under "/etc/netplan" (network manager config file, the name may vary based on your system, an example is "\01-network-manager-all.yaml"), then add the following lines at the end of the file and save it after:

```
network:
  version: 2
  ethernets:
    enx00e04c680681:
      dhcp4: no
      addresses:
        - 192.168.0.10/24
```

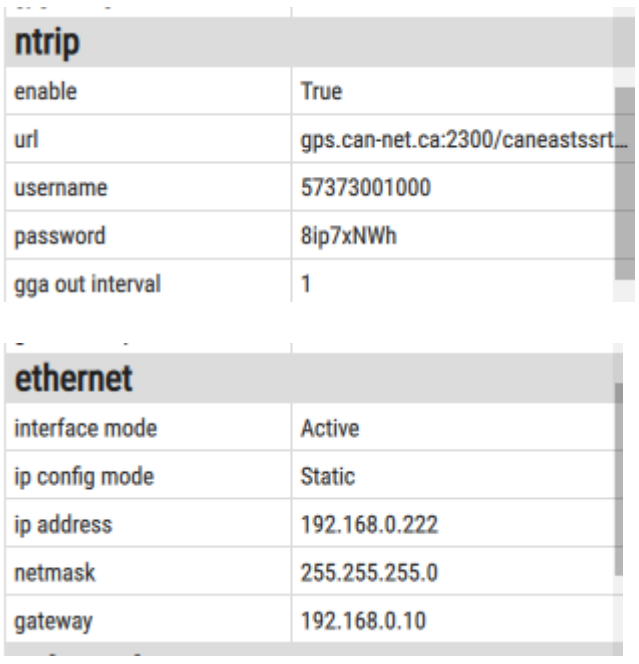
2. Configure and Monitor using Swift Console

- Open Swift console on your PC, and connect the device using the following info:



- Wait a few minutes for the GPS unit to find enough satellites in order to get an accurate localization solutions. When the solutions are found, the bottom of the console window should say **"Antenna: OK"**, along with information. Meanwhile, you should see the second LED indicator from the top on the unit chassis switch from **blinking** to **solid ON**. If the solutions are not found within a few minutes, walk around with your unit since some satellites might be blocked by high objects around you.

- After the device has found enough satellites, go to Settings tab on the left of the console window. The following two components are the most important and need to be set as follows:



ntrip	
enable	True
url	gps.can-net.ca:2300/caneastssrt...
username	57373001000
password	8ip7xNWh
gga out interval	1

ethernet	
interface mode	Active
ip config mode	Static
ip address	192.168.0.222
netmask	255.255.255.0
gateway	192.168.0.10

These settings allow the unit to access the PC network and send an NMEA GGA message regularly over the network to the NTRIP server, passing the current information of the unit in order to receive corrections. Note that the "url" entry under "ntrip" has the format of **"address:port/mountpoint"**. For the mountpoint, several streams are accessible within the Kingston area. Two of the strongest ones are *caneastssrtcm* and *caneastvrsrtcm32*. Try either one and switch to the other if the first try does not work.

3. Configure your PC to an NAT Router

Now we need to configure the PC so that it serves as a bridge/router for communication between the GPS unit and the NTRIP server over the internet.

- Enable IP forwarding

Open a terminal window and type the following command:

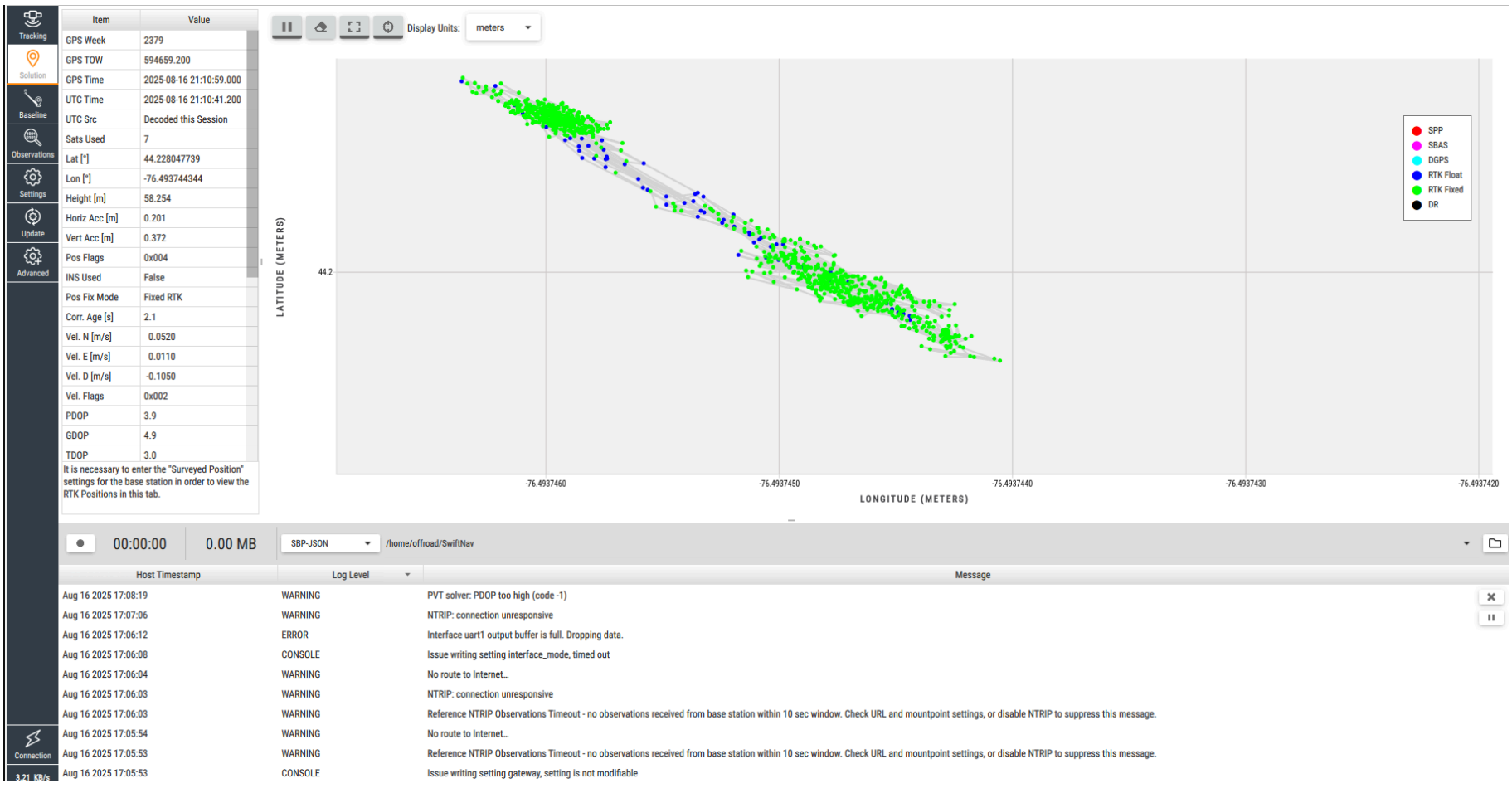
```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

- Set up NAT

Execute the following commands one-by-one:

```
sudo iptables -t nat -A POSTROUTING -o <wireless_network_interface_name> -j MASQUERADE
sudo iptables -A FORWARD -i <wireless_network_interface_name> -o <ethernet_interface_name> -m state --state
sudo iptables -A FORWARD -i <ethernet_interface_name> -o <wireless_network_interface_name> -j ACCEPT
```

With everything going well, the GPS should now be able to access real-time correction from the NTRIP server. In the Swift console, under "Solution" tab on the left, you should be able to see green dots showing on the screen with decimeter/centimeter-level drift, and "Pos Fix Mode" on the left bar should say "Fixed RTK", as shown below. Meanwhile, the third LED indicator from the top on the GPS unit chassis should either be **blinking** or **solid** red.



Follow-up: Communicating with the Receiver using ROS2

To set up communication with ROS2 for robot localization, we can use the [swiftnav-ros2] package. Most instructions are already included in its README document. One thing that we must pay attention is that according to the README file, the ros2 node expect the following SBP messages from the GPS unit:

- `UTC TIME` (ID: 259, *required/optional*) - UTC time of reported position. Required when `timestamp_source_gnss` is `True`.
- `GPS TIME` (ID: 258, *required*) - GPS time of reported position.
- `POS LLH COV` (ID: 529, *required*) - GNSS position with covariance.
- `VEL NED COV` (ID: 530, *required*) - GNSS velocity with covariance.
- `ORIENT EULER` (ID: 545, *optional*) - GNSS/INS orientation with estimated errors.
- `DOPS` (ID: 520, *optional*) - GNSS DOP (Dilution Of Precision) data.

The default setting in Swift Console, however, does not include `POS LLH COV`, `VEL NED COV` and `ORIENT EULER` to be sent. Therefore, we must manually include them by going under `Settings -> tcp server (0 or 1 depending on which port is in use) -> enabled sbp messages`, then add the missing message IDs to the list. After that, we should be able to see position, orientation etc. published over `\gpsfix` topic.

Further Applications

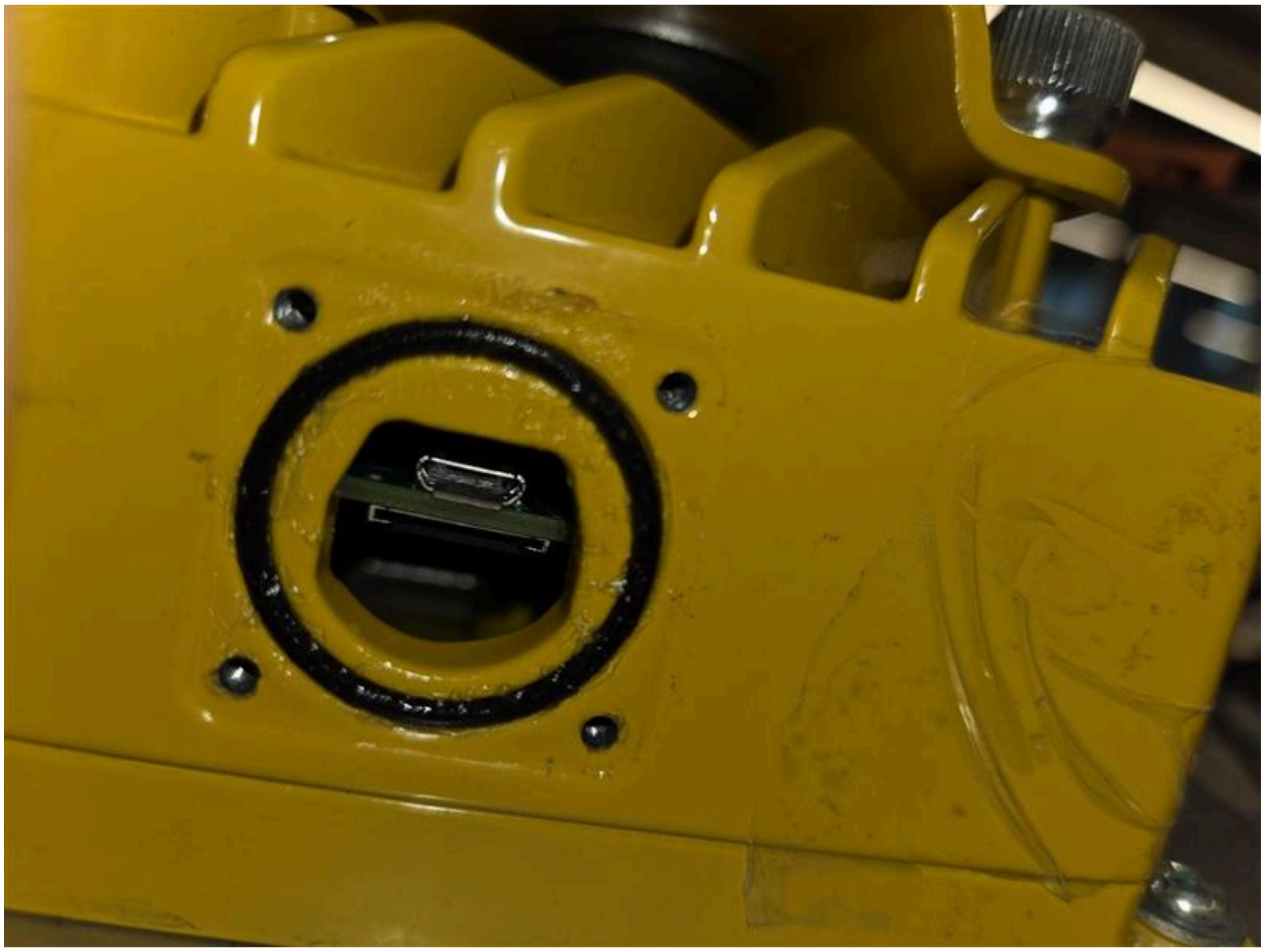
- For detailed assistance with the Swift Duro Unit, Swift Navigation has provided some useful Application Notes which contains some of the information from above but with more details.

Whoopsie Poopsies

Certain configurations, if not set right before disconnecting from the device, might block the ethernet or the serial ports. The configurations will fail to be accessed again. Some of these misconfigurations include:

- leaving "ethernet" on "Config" mode instead of "Active" mode
- leaving "ethernet" IPv4 address on "DHCP" instead of "Static"
- leaving "tcp_client 0/1" or "uart 0/1" protocols on "NMEA_OUT" instead of "SBP"
- ……

Normally, when one port is misconfigured, other ports can still be used to access the configurations. In the worst case scenario, when all available ports are misconfigured, we can open the rear cover of the chassis and get access to the backup USB-micro port inside, as shown in the figure below. This is a default-set serial port that can be connected with the baud rate of 115200. The other end of the USB cable can directly connect to the PC. Through this port, we should be able to connect to the device again using Swift Console and fix its configurations.



Comments